Project Plan Template

Agile Software Engineering Capstone

Document field	Team entry
Project title	Student Savings
Team name	Ninja Penguins
Team members	Lupe Campos Joey Contreras Doga Eski Jonathan Ochoa
Student document owner	Doga Eski
Instructor	Shah, Syed Jawad Hussain
Sponsor or client	Commerce Bank
Version and date	0.6, November 9th, 2026
Approval status	In review

Note: Consider this document as template only and feel free to customize if needed.

How to use this template

Use this document to show how the team will turn the charter, requirements, and backlog into working software during the semester. The plan should make commitments visible without pretending that early estimates will always be exact.

Replace each bracketed prompt with your team's information. Remove unused example rows and all remaining prompts before submission. Keep the document current as the project changes.

Create the first version during project planning. Review it at every sprint boundary and after major changes to scope, capacity, risks, technology, or sponsor expectations.

Revision history

Version	Date	Author	Summary of change	Reviewer or approval
0.1	9-25-2025	Lupe Campos	Initial draft	Lupe Campos
0.2	10-2-2026	Lupe Campos	Updated 1-5	Lupe Campos
0.3	10-9-2026	Lupe Campos	Updated Backlog and work breakdown	Lupe Campos
0.4	10-16-2026	Lupe Campos	Updated Risks and issues	Lupe Campos
0.5	10-26-2026	Lupe Campos	Updated Roles and Responsibilities	Lupe Campos
0.6	11-9-2026	Lupe Campos	Finalized document	Lupe Campos

Document outline

Section	Purpose
1 Plan overview	Project goal, MVP, planning approach, priorities, and constraints
2 Release strategy	Major deliverables, milestones, releases, and sprint roadmap
3 Backlog and work breakdown	Prioritized work, WBS, and work package details
4 Estimation and capacity	Effort, duration, assumptions, and available team time
5 Schedule and dependencies	Task relationships, dates, critical work, and resource balancing
6 Sprint operating plan	Sprint cadence, iteration plan, Definition of Ready, and Definition of Done
7 Roles and responsibilities	Ownership, backup coverage, and decision authority
8 Communication plan	Information needs, channels, timing, and escalation
9 Quality plan	Reviews, testing, acceptance, defects, and documentation
10 Risks and issues	Possible problems, active problems, responses, and owners
11 Change and configuration control	How the team manages scope, code, documents, and releases
12 Progress tracking	Measures, reports, forecasts, and plan updates
13 Tools and environments	Shared tools, access, and development or test environments
14 Review and submission	Approvals and final completeness check

1 Plan overview

This section gives the reader a quick picture of what the team plans to deliver and how the team will organize the work.

1.1 Product goal and minimum viable product

Student Savings will help students manage their personal finances by providing one place to track financial accounts, income, expenses, budgets, loans, and savings goals. The goal is to make it easier for students to understand their spending habits, monitor financial progress, and make informed financial decisions.

The minimum viable product (MVP) will include user registration and login, account management, transaction tracking, monthly budgets, loan tracking, savings goals, and a centralized dashboard. Users will manually enter fabricated financial information, and the application will calculate account balances and budget spending. The MVP will include the React frontend, Spring Boot backend, PostgreSQL database, and testing of the main financial calculations and user operations.

Optional features, such as CSV transaction importing, financial notifications, AI-generated summaries, and an AI financial assistant, will be considered after the core functionality is completed and tested. Direct connections to real banks and financial institutions are outside the project scope.

1.2 Planning approach and planning horizon

The team will use a combination of adaptive and predictive planning. Predictive planning will be used to establish the overall project scope, major requirements, team responsibilities, and planned milestones. Adaptive planning will be used during development so the team can adjust tasks and priorities based on progress, testing results, and issues discovered during implementation.

The team will organize development into four planned sprints. Sprint 1 focuses on authentication and user registration. Sprint 2 focuses on profile settings, financial accounts, transactions, and budgets. Sprint 3 focuses on loans, spending reviews, and savings goals. Sprint 4 focuses on proposed enhancements, integration, testing, documentation, and final preparation.

The team will prioritize detailed planning for the upcoming sprint while keeping later sprints organized around their main goals and deliverables. Tasks may be adjusted during development to address technical challenges, dependencies, and remaining time. The project manager will coordinate assignments and monitor progress, while the frontend and backend developers will communicate regularly to ensure their components integrate correctly.

1.3 Priorities constraints and assumptions

Type	Statement	Why it matters	How the team will respond
Priority	Complete the core financial features before optional features, including user authentication, account management, transaction tracking, budgets, loans, savings goals, and the dashboard.	These features provide the main purpose of Student Savings and are necessary for a useful MVP.	Prioritize required features and testing. Move optional features, such as AI assistance, notifications, and CSV importing, to later development if time is limited.
Priority	Protect users' financial information and ensure users can only access their own records.	Financial information must remain private and separate between users.	Use Spring Security, validate requests, and verify user ownership when accessing financial records. Test access restrictions between different users.
Constraint	The project uses React for the frontend, Spring Boot for the backend, and PostgreSQL 16 for the database.	Changing the technology stack would require significant development changes and could delay integration.	Continue developing with the existing technologies and coordinate frontend and backend changes through the shared GitHub repository.
Constraint	The application uses Docker Compose to run the frontend, backend, and database as separate services.	The services depend on one another and must communicate correctly for the application to work.	Maintain the existing Docker Compose configuration and test that all three services start and communicate correctly.
Constraint	The project uses fabricated financial information and does not include direct connections to real banks or financial institutions.	Real banking integration would introduce additional security, privacy, and development requirements outside the current project scope.	Keep financial information manually entered and use fabricated data for development, testing, and demonstrations.

Type	Statement	Why it matters	How the team will respond
Assumption	Users will manually enter their financial accounts, transactions, budgets, loans, and savings goals.	The backend and frontend must support users entering and updating their own financial information.	Provide forms and CRUD operations for the required financial features. Reassess CSV importing as an optional enhancement.
Assumption	The frontend and backend will communicate through REST API requests using HTTP and JSON.	Both developers need consistent API endpoints and data formats for the application to function correctly.	Coordinate endpoint names, request formats, and response formats between frontend and backend development. Test each integration as features are completed.
Assumption	The application will be developed and demonstrated using the team's local Docker environment.	All team members need a consistent environment to run and test the application.	Use the shared Docker Compose configuration and verify that each team member can build and run the application locally.
Assumption	The team will have enough time to complete the required features and test the integrated application within the project schedule.	Delays in backend or frontend development could affect the final deliverable.	Monitor sprint progress, prioritize unfinished required features, and reduce optional work when necessary.

2 Release strategy

A release plan shows the overall path to the final product. It stays less detailed than an individual sprint plan.

2.1 Major deliverables and milestones

ID	Deliverable or milestone	Evidence	Target date or sprint	Owner	Reviewer
M-01	Project architecture and design completed	Architecture document, ADRs, system diagrams, and data model	Sprint 1	Jonathan Ochoa	Entire team and instructor
M-02	Authentication and registration completed	Working registration, login, logout, and password reset functionality; backend and frontend code in GitHub	Sprint 1	Joey Contreras	Doga Eski and Jonathan Ochoa
M-03	Financial accounts and budgets implemented	Working account creation and management, transaction functionality, and monthly budget features	Sprint 2	Joey Contreras	Doga Eski and Jonathan Ochoa
M-04	Profile and settings implemented	Working profile and settings interface connected to the backend	Sprint 2	Doga Eski	Joey Contreras and Jonathan Ochoa
M-05	Loans and spending review implemented	Working loan management, payment tracking, and spending review functionality	Sprint 3	Joey Contreras / Doga Eski	Jonathan Ochoa
M-06	Savings goals implemented	Working goal creation, progress tracking, and target date functionality	Sprint 3	Lupe Campos	Joey Contreras and Doga Eski

ID	Deliverable or milestone	Evidence	Target date or sprint	Owner	Reviewer
M-07	Integrated application and dashboard completed	Working React frontend, Spring Boot backend, PostgreSQL database, and dashboard displaying financial information	Sprint 3	Joey Contreras / Doga Eski	Entire team
M-08	Optional features evaluated	Documentation or working prototypes for CSV importing, notifications, and AI financial features, if time permits	Sprint 4	Entire team	Jonathan Ochoa
M-09	Final testing and integration completed	Test results, bug fixes, working Docker Compose environment, and successful application demonstration	Sprint 4	Joey Contreras / Doga Eski	Entire team
M-10	Final project documentation and release completed	Updated architecture, project documentation, GitHub repository, and final presentation/demo	Sprint 4	Lupe Campos	Jonathan Ochoa and instructor

2.2 Release and sprint roadmap

What to write: *List one clear goal and one demonstrable outcome for each sprint. Include dependencies and review evidence. Avoid using the roadmap as a list of individual assignments.*

Sprint	Sprint goal	Planned demonstrable outcome	Main dependencies	Review evidence
Sprint 0	Establish the project foundation and prepare the development environment.	Project charter, requirements, initial architecture, shared GitHub repository, React frontend, Spring Boot backend, PostgreSQL database, and Docker Compose configuration are prepared.	Team assignments, project requirements, technology selection, and repository access.	Reviewed project documents, GitHub repository, and successful Docker Compose startup.
Sprint 1	Establish the foundation for secure user access.	Users can register, sign in, sign out, and access protected application functionality.	Sprint 0 setup, database connection, frontend/backend integration, and authentication design.	Live registration and login demonstration, authentication tests, and GitHub code review.
Sprint 2	Allow users to manage their financial information and monthly spending.	Users can manage financial accounts, record income and expenses, create monthly budgets, and manage profile settings.	Working authentication, database persistence, account and transaction functionality, and REST API integration.	Demonstration of account and transaction management, budget calculations, profile functionality, and test results.
Sprint 3	Expand financial tracking and provide a complete financial overview.	Users can manage loans, track payments, create savings goals, review spending, and view their financial information through a centralized dashboard.	Completed account and transaction functionality, budget data, database relationships, and dashboard API integration.	Live demonstration of loans, goals, spending review, dashboard information, and financial calculation tests.

Sprint	Sprint goal	Planned demonstrable outcome	Main dependencies	Review evidence
Sprint 4	Finalize, test, and prepare the application for delivery.	An integrated Student Savings application with completed core features, corrected defects, updated documentation, and optional enhancements if time permits.	Completion of required features, stable frontend/backend integration, and available testing environment.	Final application demonstration, test results, Docker Compose verification, updated GitHub repository, and final documentation.

3 Backlog and work breakdown

3.1 Prioritized backlog summary

Backlog ID	Epic story or deliverable	Priority	Estimate	Target sprint	Status
TECH-01	Set up React frontend, Spring Boot backend, PostgreSQL database, and Docker Compose environment.	Must	TBD	Sprint 0	Ready
UC-AUTH-02	As an unregistered user, I can create an account so I can use Student Savings.	Must	TBD	Sprint 1	Proposed
UC-AUTH-03	As a registered user, I can sign in securely to access my financial information.	Must	TBD	Sprint 1	Proposed
UC-AUTH-04	As a registered user, I can sign out to protect my account.	Must	TBD	Sprint 1	Proposed
UC-AUTH-01	As a registered user, I can reset my password if I forget it.	Must	TBD	Sprint 1	Proposed

Backlog ID	Epic story or deliverable	Priority	Estimate	Target sprint	Status
UC-PROF-01	As a registered user, I can manage my profile and settings.	Must	TBD	Sprint 2	Proposed
UC-ACC-01	As a registered user, I can create, view, update, and delete financial accounts.	Must	TBD	Sprint 2	Proposed
UC-ACC-02	As a registered user, I can record and manage income and expenses associated with my accounts.	Must	TBD	Sprint 2	Proposed
UC-BUD-01	As a registered user, I can create monthly budgets by category and track my spending.	Must	TBD	Sprint 2	Proposed
UC-LOAN-01	As a registered user, I can add and manage loans to track what I owe.	Must	TBD	Sprint 3	Proposed
UC-LOAN-02	As a registered user, I can track loan payments, remaining balances, and upcoming due dates.	Must	TBD	Sprint 3	Proposed
UC-UC-SPEND-01	As a registered user, I can review my spending to understand where my money goes.	Must	TBD	Sprint 3	Proposed
UC-GOAL-01	As a registered user, I can create savings goals and track my progress.	Must (Stretch)	TBD	Sprint 3	Proposed
UC-GOAL-02	As a registered user, I can follow a savings plan to work toward my financial goals.	Should	TBD	Sprint 4	Proposed

Backlog ID	Epic story or deliverable	Priority	Estimate	Target sprint	Status
UC-NOTIF-01	As a registered user, I can receive financial notifications and reminders.	Should	TBD	Sprint 4	Proposed
UC-AI-01	As a registered user, I can generate an AI financial summary of my financial activity.	Must (Stretch)	TBD	Sprint 4	Proposed
UC-AI-02	As a registered user, I can use a financial assistant for help understanding my finances.	Must (Stretch)	TBD	Sprint 4	Proposed
UC-SUP-01	As a registered user, I can get help and submit feedback about the application.	Should	TBD	Sprint 4	Proposed
UC-ADMIN-01	As a system administrator, I can manage users and feedback.	Should	TBD	Sprint 4	Proposed
TECH-02	Complete integration, test core financial calculations, fix defects, and prepare the final application.	Must	TBD	Sprint 4	Proposed
FUT-01	Allow users to import transactions using CSV files.	Could	TBD	Future / If time permits	Proposed

3.2 Work breakdown structure

1.0 Project foundation

- **1.1 Establish project requirements and scope.**
- **1.2 Create the shared GitHub repository.**
- **1.3 Set up the React frontend.**
- **1.4 Set up the Spring Boot backend.**
- **1.5 Configure the PostgreSQL database.**

- 1.6 Configure Docker Compose for the frontend, backend, and database.
- 1.7 Verify that the development environment can build and run.

Deliverable: A shared development environment with the frontend, backend, and database configured.

2.0 User authentication and access

- 2.1 Define registration and login requirements.
- 2.2 Design the user data model.
- 2.3 Implement user registration.
- 2.4 Implement login functionality.
- 2.5 Implement logout functionality.
- 2.6 Implement password reset functionality.
- 2.7 Configure authentication and authorization.
- 2.8 Test registration, login, logout, and protected access.

Deliverable: Users can register, sign in, and securely access their accounts.

3.0 Financial account management

- 3.1 Define checking, savings, and credit card account requirements.
- 3.2 Design the account data model.
- 3.3 Implement account creation.
- 3.4 Implement account retrieval and display.
- 3.5 Implement account updates and deletion.
- 3.6 Validate account names and account types.
- 3.7 Test account management.

Deliverable: Users can manage their financial accounts.

4.0 Transaction management

- 4.1 Define income and expense requirements.
- 4.2 Design the transaction data model.
- 4.3 Implement transaction creation.
- 4.4 Implement transaction retrieval and display.
- 4.5 Implement transaction updates and deletion.
- 4.6 Associate transactions with the correct user-owned accounts.
- 4.7 Implement account balance calculations.
- 4.8 Test income, expenses, and balance calculations.

Deliverable: Users can record and manage transactions, and account balances reflect their transactions.

5.0 Budget management

- 5.1 Define monthly budget requirements.
- 5.2 Design the budget data model.
- 5.3 Implement budget creation.
- 5.4 Implement budget updates and deletion.
- 5.5 Calculate spending by category and month.

- 5.6 Display budget usage.
- 5.7 Test budget calculations.

Deliverable: Users can create monthly budgets and monitor spending.

6.0 Loan management

- 6.1 Define loan tracking requirements.
- 6.2 Design the loan data model.
- 6.3 Implement loan creation and management.
- 6.4 Store original and remaining loan balances.
- 6.5 Track monthly payments and due dates.
- 6.6 Calculate total remaining loan balances.
- 6.7 Identify upcoming payment dates.
- 6.8 Test loan tracking calculations.

Deliverable: Users can track loans, balances, and upcoming payments.

7.0 Savings goal management

- 7.1 Define savings goal requirements.
- 7.2 Design the goal data model.
- 7.3 Implement goal creation.
- 7.4 Implement goal updates and deletion.
- 7.5 Track saved amounts and target amounts.
- 7.6 Display progress toward savings goals.
- 7.7 Test savings goal functionality.

Deliverable: Users can create savings goals and monitor their progress.

8.0 Dashboard and spending review

- 8.1 Define dashboard information requirements.
- 8.2 Design the dashboard response structure.
- 8.3 Implement the dashboard backend endpoint.
- 8.4 Retrieve account information.
- 8.5 Retrieve transaction information.
- 8.6 Retrieve budget, loan, and savings goal information.
- 8.7 Connect the React dashboard to the backend.
- 8.8 Test dashboard information and calculations.

Deliverable: Users can view their financial information through one centralized dashboard.

9.0 Integration and final release

- 9.1 Integrate frontend and backend functionality.
- 9.2 Verify database persistence.
- 9.3 Test user authentication and data access restrictions.
- 9.4 Test account, transaction, budget, loan, and goal operations.
- 9.5 Fix identified defects.

- 9.6 Verify Docker Compose startup and service communication.
- 9.7 Update project and architecture documentation.
- 9.8 Prepare the final application demonstration.

Deliverable: An integrated, tested Student Savings application ready for final demonstration.

3.3 Work package details

WBS ID	Work package	Owner	Output and completion test	Effort	Duration	Predecessor
1.3	Configure React frontend	Doga Eski	Frontend builds and starts successfully; main application interface loads.	6 hours	2 days	1.2
1.4	Configure Spring Boot backend	Joey Contreras	Backend builds and starts successfully; API responds to requests.	8 hours	3 days	1.2
1.5	Configure PostgreSQL database	Joey Contreras	Database starts and backend successfully connects to PostgreSQL.	5 hours	2 days	1.4
1.6	Configure Docker Compose services	Joey Contreras	Frontend, backend, and database start together using Docker Compose.	5 hours	2 days	1.3, 1.4, 1.5
2.3	Implement user registration API	Joey Contreras	Valid users can register and user records are stored securely.	10 hours	3 days	1.4, 1.5
2.4	Implement login and authentication	Joey Contreras	Registered users can sign in and protected endpoints require authentication.	12 hours	4 days	2.3
2.5	Implement frontend authentication pages	Doga Eski	Registration and login pages connect successfully to backend endpoints.	10 hours	3 days	1.3, 2.3, 2.4

WBS ID	Work package	Owner	Output and completion test	Effort	Duration	Predecessor
3.3	Implement financial account management	Joey Contreras	Users can create, retrieve, update, and delete their own accounts.	12 hours	4 days	2.4, 1.5
4.3	Implement transaction management	Joey Contreras	Users can record, retrieve, update, and delete transactions associated with their accounts.	14 hours	5 days	3.3
4.7	Implement account balance calculations	Joey Contreras	Account balance equals opening balance plus associated transactions; calculations are tested.	6 hours	2 days	4.3
5.3	Implement monthly budget management	Joey Contreras	Users can create and manage monthly category budgets.	10 hours	3 days	4.3
5.5	Implement budget spending calculations	Joey Contreras	Spending totals correctly include applicable negative transactions for the selected month and category.	8 hours	3 days	5.3, 4.3
6.3	Implement loan management	Joey Contreras	Users can add, retrieve, update, and delete loan records.	10 hours	3 days	2.4, 1.5
7.3	Implement savings goal management	Joey Contreras	Users can create and update savings goals and track saved amounts.	8 hours	3 days	2.4, 1.5
8.3	Implement centralized dashboard API	Joey Contreras	Dashboard endpoint returns the user's financial information in the agreed response structure.	12 hours	4 days	3.3, 4.3, 5.3, 6.3, 7.3

WBS ID	Work package	Owner	Output and completion test	Effort	Duration	Predecessor
8.7	Connect React dashboard to backend	Doga Eski	Dashboard displays the information returned by the backend without requiring duplicate frontend calculations.	10 hours	3 days	8.3
9.1	Integrate frontend and backend features	Joey Contreras / Doga Eski	Required application features communicate correctly across the frontend, backend, and database.	12 hours	4 days	3.3, 4.3, 5.3, 6.3, 7.3, 8.7
9.4	Test core financial functionality	Joey Contreras / Doga Eski	Account, transaction, budget, loan, and goal operations produce expected results.	12 hours	4 days	9.1
9.5	Fix defects found during testing	Joey Contreras / Doga Eski	Identified defects are corrected and affected functionality is retested.	10 hours	3 days	9.4
9.7	Complete project and architecture documentation	Lupe Campos	Architecture, requirements, implementation details, and final project documentation are updated.	10 hours	4 days	9.1
9.8	Coordinate final demonstration and release	Jonathan Ochoa	Final demonstration is prepared, project progress is reviewed, and deliverables are ready for submission.	6 hours	2 days	9.5, 9.7

4 Estimation and capacity

4.1 Estimation method and assumptions

The Student Savings team will use a combination of story-point estimates and person-hour estimates to plan development work. Story points will be used to estimate the relative size and complexity of backlog items, while person-hours will be used to estimate individual work packages and assign development tasks.

Estimates will consider the complexity of the task, the technologies being used, dependencies between the frontend and backend, testing requirements, and the time needed for documentation and review. The team will review and update estimates during sprint planning and compare estimated effort with actual effort at the end of each sprint.

The following assumptions will guide estimation:

- *Team experience: Estimates assume team members have sufficient familiarity with React, Java, Spring Boot, PostgreSQL, GitHub, and Docker to complete their assigned tasks.*
- *Development environment: The existing GitHub repository and Docker Compose configuration will provide the shared starting point for development and testing.*
- *Technology stack: The team will continue using React, Spring Boot, and PostgreSQL rather than changing major technologies during development.*
- *Task dependencies: Backend functionality and database changes must be available before related frontend features can be fully integrated.*
- *Testing: Completion includes basic functionality testing, validation of expected results, and correction of identified defects.*
- *Code review: Team members will review important changes before integrating them into the shared GitHub repository.*
- *Documentation: Major features and architecture changes will be documented as development progresses.*
- *Available capacity: Estimates assume team members have enough time outside other academic responsibilities to complete their assigned work.*
- *Optional features: CSV importing, AI assistance, and notifications will only be scheduled if the required MVP functionality is completed.*

A work package will be considered complete when its expected output has been implemented, the completion test has passed, related changes have been integrated into GitHub, and necessary documentation has been updated..

4.2 Effort and duration

Effort means labor time. Duration means calendar time. A task may require two hours of effort but three days of duration while the team waits for a review or external decision.

Task	Effort estimate	Duration estimate	Reason for difference	Confidence
Configure frontend, backend, and database	19 person-hours	5 working days	Services must be configured and tested together.	Medium
Implement user registration and authentication	32 person-hours	7 working days	Requires backend security, database integration, and frontend connection.	Medium
Implement financial account management	12 person-hours	4 working days	Requires account data modeling, API functionality, and frontend integration.	Medium
Implement transaction management	14 person-hours	5 working days	Requires account relationships, transaction validation, and CRUD operations.	Medium
Implement account balance calculations	6 person-hours	2 working days	Requires transaction data to be available and calculations to be verified.	High
Implement monthly budgets and spending calculations	18 person-hours	6 working days	Requires transaction data, category matching, and monthly calculations.	Medium
Implement loan management	10 person-hours	3 working days	Requires loan records, payment information, and due-date tracking.	Medium
Implement savings goals	8 person-hours	3 working days	Requires goal records and progress calculations.	Medium

Task	Effort estimate	Duration estimate	Reason for difference	Confidence
Implement centralized dashboard	22 person-hours	7 working days	Requires multiple financial features to be available before integration.	Medium
Complete frontend/backend integration	12 person-hours	4 working days	Requires coordination between developers and resolving API compatibility issues.	Medium
Complete testing and defect correction	22 person-hours	7 working days	Testing may reveal problems that require additional development and retesting.	Medium
Complete final documentation and demonstration	16 person-hours	5 working days	Requires reviewing implementation, updating documents, and preparing the presentation.	Medium

4.3 Team capacity

Assuming a one-week sprint, the team will reserve approximately 75% of available time for productive project work. The remaining time accounts for meetings, communication, reviews, interruptions, and other responsibilities.

Team member	Role	Hours available per week	Planned absence	Focus factor	Usable sprint capacity	Backup
Team member	**Role**	**Hours available per week**	**Planned absence**	**Focus factor**	**Usable sprint capacity**	**Backup**
Joey Contreras	Backend Developer	8	None reported	75%	6 hours	Jonathan Ochoa

Team member	Role	Hours available per week	Planned absence	Focus factor	Usable sprint capacity	Backup
Doga Eski	Frontend Developer	8	None reported	75%	6 hours	Lupe Campos
Lupe Campos	Documentation	6	None reported	75%	4.5 hours	Jonathan Ochoa
Jonathan Ochoa	Project Manager	6	None reported	75%	4.5 hours	Lupe Campos

5 Schedule and dependencies

5.1 Task dependencies

From task	To task	Dependency type	Explanation	Risk if delayed
From task	**To task**	**Dependency type**	**Explanation**	**Risk if delayed**
Requirements documentation	Architecture and database design	Finish to start	The team needs agreed requirements before finalizing the architecture and data structure.	Architecture and implementation may need rework.
Database design	Registration API	Finish to start	Registration requires an agreed user data structure and database connection.	User authentication work slips.
Backend API development	Frontend integration	Finish to start	The frontend needs working API endpoints and agreed request/response formats.	Frontend functionality cannot be fully integrated.

From task	To task	Dependency type	Explanation	Risk if delayed
React frontend setup	Login page implementation	Finish to start	The frontend environment and components must be available before implementing the login interface.	Authentication interface development is delayed.
Registration API	Login functionality	Finish to start	Login depends on users being registered and stored in the database.	Users cannot successfully sign in.
Authentication implementation	Account management	Finish to start	Financial account operations must be associated with an authenticated user.	Account features may not properly protect user information.
Account management	Transaction management	Finish to start	Transactions must be associated with an existing financial account.	Transaction creation and balance calculations are delayed.
Transaction management	Account balance calculations	Finish to start	Balance calculations require transaction records and their associated accounts.	Displayed account balances may be inaccurate or incomplete.
Transaction management	Budget spending calculations	Finish to start	Budget usage depends on transaction amounts, categories, and dates.	Monthly budget totals cannot be calculated correctly.

From task	To task	Dependency type	Explanation	Risk if delayed
Loan management	Dashboard integration	Finish to start	The dashboard needs loan information available from the backend.	Dashboard may display incomplete financial information.
Savings goal management	Dashboard integration	Finish to start	The dashboard needs savings goal information to display progress.	Savings goal information may be missing from the dashboard.
Backend dashboard API	Frontend dashboard integration	Finish to start	The React dashboard needs the backend response structure to retrieve and display financial information.	Dashboard integration and testing are delayed.
Docker Compose configuration	Integrated application testing	Finish to start	The frontend, backend, and database must run together before the complete environment can be tested.	Integration testing and demonstration may be delayed.
Feature implementation	Feature testing	Start to start	Testing preparation can begin while features are being implemented, allowing test cases to be developed early.	Defects may be discovered later in development.
Frontend/backend integration	Final system testing	Finish to start	Core components must be connected before complete application testing can occur.	Final testing and release preparation slip.

From task	To task	Dependency type	Explanation	Risk if delayed
Final system testing	Defect correction	Finish to start	Testing results identify the problems that need to be corrected.	Known defects remain in the final application.
Defect correction	Final demonstration	Finish to start	Critical issues should be resolved before the application is demonstrated.	The final demonstration may contain incomplete or broken functionality.
Architecture and project documentation	Final project submission	Finish to start	Required documentation must be reviewed and completed before final submission.	Project deliverables may be incomplete.

5.2 Schedule

Ninja Penguins Project visibility

5.3 Critical work and float

What to write: *Identify tasks that directly control a major deadline. Record any available slack and the response if critical work slips.*

Critical task or path	Deadline controlled	Available slack	Early warning	Response
Requirements and architecture finalized	Sprint 1 completion	1–2 days	Requirements or database design are still changing near the end of the sprint	Finalize the required MVP scope and defer nonessential changes

Critical task or path	Deadline controlled	Available slack	Early warning	Response
Database setup to registration and login functionality	Sprint 1 authentication milestone	1 day	Registration or login does not work by the sprint midpoint	Have the backend and frontend developers work together on the minimum working authentication flow
Account management to transaction tracking	Sprint 2 feature completion	1–2 days	Accounts cannot be created or transactions cannot be saved	Prioritize account and transaction CRUD; postpone optional features
Transaction data to budget calculations	Sprint 2 feature completion	1 day	Transaction categories or dates are not available for budget totals	Fix transaction data handling before adding optional budget enhancements
Backend APIs to dashboard integration	Sprint 3 dashboard milestone	1–2 days	Dashboard data is unavailable or does not match the frontend's expected structure	Agree on the API response format and use sample data while resolving integration issues
Loans and savings goals to dashboard	Sprint 3 feature completion	1 day	Loan or goal information is missing from the dashboard	Complete the minimum required data and defer advanced calculations
Docker Compose integration and system testing	Final demo and release	1 day	One or more services fail to start or communicate	Prioritize fixing container configuration and test the complete application locally
Final testing and defect correction	Final demonstration and submission	1–2 days	Critical bugs remain during the final testing period	Fix critical defects first and defer low-priority enhancements

Critical task or path	Deadline controlled	Available slack	Early warning	Response
Documentation and demo preparation	Final project submission	1 day	Screenshots, architecture details, or presentation materials are incomplete	Divide remaining documentation tasks among team members and focus on required deliverables

5.4 Resource balancing

This is how we are balancing our work and planning for backups in case of unforeseen events.

Team member	Primary role	Charter responsibility	Decision authority or backup
Jonathan	Project Manager	Maintains direction and scope of project	Main decision maker
Doga	Front-end development	Front end development of project	Final decision on UI of our app.
Joey	Back-end development	Development of back-end and anything managing our user's data.	Final decision on data management in our app.
Lupe	Documentation	Documenting anything related to the project.	Final decision on documentation and back up for the front-end development.

6 Sprint operating plan

6.1 Sprint cadence

Event	Purpose	Participants	Timing and timebox	Output
Sprint planning	Choose a sprint goal and work that fits capacity	Whole team	First day, 60 minutes	Sprint goal and sprint backlog
Sprint planning	Choose the sprint goal, prioritize tasks, and assign work based on team capacity	Whole team	First day of each weekly sprint, 60 minutes	Sprint goal and sprint backlog
Standup	Share progress, identify blockers, and coordinate backend, frontend, and documentation work	Whole team	Daily during the sprint, 10–15 minutes	Updated task board and follow-up actions

Event	Purpose	Participants	Timing and timebox	Output
Review or demo	Demonstrate completed features, review progress, and gather feedback	Team and project reviewers	Last day of each weekly sprint, 30–45 minutes	Feature feedback, acceptance decisions, and updated backlog
Retrospective	Discuss what went well, identify problems, and decide how to improve teamwork	Student team	After the sprint review, 15–30 minutes	One or two improvement actions for the next sprint

6.2 Sprint plan

Sprint field	Team entry
Sprint number and dates	Final Sprint & Demo week
Sprint goal	Complete planned feature updates, verify that the application works, and prepare a working Student Savings demonstration.
Committed stories or deliverables	Dashboard updates and additional functionality; user profile and settings (UC-PROF-01); AI feature work (UC-AI-01 / UC-AI-02, if time permits); Docker environment and backend fixes; final demo presentation.
Capacity and known absences	Exact available person-hours were not recorded. The team must account for time limitations and personal commitments when assigning remaining work.
Main dependencies	Existing login and database setup; completed dashboard design; backend fixes; Docker Compose environment; coordination between frontend and backend developers; completed documentation and presentation materials.
Review evidence	Working application demonstration, feature testing results, Docker Compose startup, updated dashboard, and demo presentation.
Plan adjustment rule	Prioritize fixing critical issues and completing the core demonstration. If time becomes limited, defer optional AI enhancements or lower-priority features rather than delaying demo readiness.

6.3 Definition of Ready and Definition of Done

Definition	Required conditions
Ready	User value and requirements are clear; acceptance criteria are documented; dependencies are identified; the task is assigned to a team member; the work is small enough to complete within a one-week sprint; and the team knows how the feature will be tested.
Done	Acceptance criteria are met; code is reviewed and merged into the GitHub repository; relevant tests pass; frontend and backend functionality work together; defects are documented; related project documentation is updated; and the feature can be demonstrated.
Team Ready definition	The team agrees that a task has a clear purpose, documented requirements, an assigned owner, identified dependencies, and a realistic one-week completion target. Required tools, environments, and resources must be available before development begins.
Team Done definition	The assigned feature works as expected, meets its acceptance criteria, and has been tested. Code changes are committed and merged through GitHub, integration issues are addressed, known defects are recorded, and necessary documentation is updated. The completed work must be ready for review or demonstration.

7 Roles and responsibilities

Role or person	Main responsibilities	Decision authority	Backup	Expected evidence
Jonathan	Project Manager	Maintains direction and scope of project	Main decision maker	Sprint plans, updated task board, meeting notes, project schedule, progress reports, risk tracking, and final demo presentation.

Role or person	Main responsibilities	Decision authority	Backup	Expected evidence
Doga	Front-end development	Front end development of project	Final decision on UI of our app.	React source code, login and dashboard screenshots, user profile/settings pages, Figma designs, frontend test results, and working UI demonstrations.
Joey	Back-end development	Development of back-end and anything managing our user's data.	Final decision on data management in our app.	Spring Boot source code, REST API endpoints, database configuration, authentication implementation, backend test results, and evidence of successful frontend/backend integration.
Lupe	Documentation	Documenting anything related to the project.	Final decision on documentation and back up for the front-end development.	Completed project charter, requirements document, project plan, system architecture documentation, updated use cases, and final project documentation.

8 Communication plan

Information	Creator or source	Audience	Method or location	Frequency or trigger	Owner
Sprint status	Project team	Team and instructor	Project visibility sheet	End of each sprint	Professor
Technical decision	Team discussion	Team	Through teams chat/ meeting	When a significant decision occurs	Frontend or backend dependin g on case

8.1 Team communication rules

Any issues or concerns must be addressed with respect through our teams chat or our meetings

9 Quality plan

Quality activity	When it occurs	Owner	Entry condition	Completion evidence
Code review	Before merge	Peer reviewer	Code changes are completed and automated checks pass	Approved GitHub pull request
Unit testing	During implementation	Story owner / Backend developer	Testable behavior is identified	Passing unit tests and recorded results
Integration or UI testing	Before sprint review or release	Frontend and backend developers	Increment is deployed to the test environment	Recorded test results, screenshots, and documented defects
Requirements verification	During development and before sprint review	Project manager and documentati on lead	Requirements and acceptance criteria are documented	Requirements checklist showing completed and outstanding items
Database testing	During backend development and integration	Backend developer	Database schema and connections are configured	Successful database operations and test results

Quality activity	When it occurs	Owner	Entry condition	Completion evidence
Docker environment testing	Before integration testing and demonstration	Backend developer	Docker Compose configuration is available	Evidence that frontend, backend, and database containers start and communicate successfully
Usability testing	Before sprint review and final demo	Frontend developer and project manager	Working interface is available	User feedback, screenshots, and documented usability issues
Documentation review	Before final submission	Documentation lead	Project documents and implementation details are available	Reviewed and updated project charter, requirements, architecture, and project plan
Final acceptance testing	Before final project demonstration	Whole team	Core features are integrated and critical defects are resolved	Completed acceptance checklist and successful application demonstration

9.1 Defect handling

Defects will be classified as critical, high, medium, or low based on their impact on functionality, security, and usability, with the project manager coordinating triage and the appropriate developer responsible for fixing them. Critical defects that prevent core features from working or compromise user data must be resolved before a sprint is accepted or the project is released, while unresolved lower-priority defects will be documented in GitHub with their severity, status, and assigned owner.

10 Risks and issues

10.1 Risk register

Risk ID	Possible problem	Probability	Impact	Trigger	Mitigation and contingency	Owner
R-01	Authentication integration takes longer than planned	Medium	High	Login flow does not work by sprint midpoint	Pair on a minimal library-supported flow; defer optional identity features	Backend lead

Risk ID	Possible problem	Probability	Impact	Trigger	Mitigation and contingency	Owner
R-02	Team members are unfamiliar with technologies or tools	High	Medium	Development tasks take longer because of troubleshooting or learning	Review documentation, share knowledge, and ask teammates for assistance; simplify implementation if needed	Whole team
R-03	Poor time management delays project tasks	Medium	High	Planned tasks are not completed by the end of a sprint	Set weekly priorities and monitor progress; move lower-priority tasks to a later sprint	Project manager
R-04	Dashboard development takes longer than expected	High	High	Dashboard features are incomplete by the sprint deadline	Prioritize essential dashboard functions and postpone optional visual enhancements	Frontend developer
R-05	Personal events reduce team availability	Medium	High	Team members miss planned work or deadlines	Communicate availability early and redistribute tasks among available members	Project manager
R-06	Docker migration or backend issues affect integration	Medium	High	Frontend, backend, or database containers fail to communicate	Test services individually and fix configuration issues; use local development as a temporary fallback	Backend developer
R-07	Optional AI features take time away from core functionality	Medium	Medium	AI development delays required dashboard or financial features	Complete core features first and defer AI enhancements if necessary	Project manager and development team

10.2 Issue log

Issue ID	Current problem	Effect	Action	Owner	Due date	Status
I-01	Team members experienced unfamiliarity with project tools and technologies	Slower initial progress and additional learning time	Review documentation, share knowledge, and divide tasks based on experience	Whole team	9/25/2026	In progress
I-02	Time management challenges affected early project planning	Delays in completing project documentation and development tasks	Set weekly priorities and monitor task completion	Project manager	9/25/2026	In progress
I-03	Dashboard setup and development took too much time	Delayed progress on dashboard functionality	Prioritize essential dashboard features and postpone optional enhancements	Frontend developer	10/23/2026	In progress
I-04	Personal events affected team members' availability	Reduced development time and possible delays in assigned tasks	Communicate schedule conflicts and redistribute work when necessary	Project manager	10/23/2026	In progress
I-05	Backend issues occurred during development and Docker migration	Delayed application integration and testing	Troubleshoot backend issues and verify communication between containers	Backend developer	10/23/2026	In progress

Issue ID	Current problem	Effect	Action	Owner	Due date	Status
I-06	Final planned features needed additional updates before demonstration	Risk of incomplete features during demo week	Prioritize required functionality, test the application, and fix critical defects	Whole team	10/30/2026	In progress

11 Change and configuration control

11.1 Scope and plan change process

Step	Team procedure
1 Request	Record the requested change, reason, requester, and date in the project task board or change log.
2 Analyze	Evaluate the change's value, estimated effort, schedule impact, risks, quality impact, and effect on project documentation.
3 Decide	The project manager (Jonathan Ochoa) approves routine changes, while major changes affecting project scope, deadlines, or core requirements must be discussed and approved by the entire team.
4 Update	Update the GitHub backlog, requirements, project plan, architecture documents, tests, and decision log as needed.
5 Communicate	Notify affected team members and stakeholders about the approved change, its reason, and any changes to responsibilities or deadlines.

11.2 Configuration management

The team uses GitHub to store and manage the Student Savings source code, including the React frontend, Spring Boot backend, Docker configuration, and project documentation, with changes committed to the repository and reviewed before merging into the main branch. Project documents are stored in the repository, and the team should use consistent commit messages, version tags for releases, and agreed-upon access permissions to maintain version history and protect the project's approved baseline.

12 Progress tracking

Measure	How calculated or judged	Source	Frequency	Decision it supports
Sprint goal result	Met, partially met, or not met with explanation	Sprint review	Each sprint	Adjust scope or process

Measure	How calculated or judged	Source	Frequency	Decision it supports
Forecast	Expected completion based on remaining work and observed capacity	Backlog and capacity	Each sprint	Move or split work
Test and defect status	Required tests passing and open defects by severity	Test results and GitHub issue tracker	At least weekly	Release readiness
GitHub development activity	Review commits, merged changes, and updates to frontend and backend files	GitHub commit history and repository	Weekly	Identify development progress and delays
Frontend progress	Check completion of React pages, components, dashboard, and user interface requirements	GitHub frontend folder and UI demonstrations	Weekly	Identify unfinished frontend features
Backend progress	Check Spring Boot functionality, API development, database integration, and backend changes	GitHub backend folder and test results	Weekly	Identify backend development or integration problems
Environment and integration status	Verify that Docker Compose starts the frontend, backend, and database and that services communicate	`Dockerfile`, `compose.yaml`, and integration tests	Weekly and before demo	Determine whether the application is ready for integration or demonstration
Documentation progress	Check whether the project charter, requirements, architecture, and project plan are complete and updated	GitHub project documents	Each sprint	Identify missing documentation before submission

Measure	How calculated or judged	Source	Frequency	Decision it supports
Final demonstration readiness	Verify that core features work together and can be demonstrated without critical defects	Application demo and acceptance checklist	Before final demo	Approve final demonstration and submission

12.1 Status report format

The team will provide a weekly status report summarizing completed work, upcoming tasks, and any blockers affecting progress in the project visibility sheet. Each report will also include changes in project risks, testing results, defect status, and whether the project is on schedule. Any decisions needed regarding deadlines, task priorities, or project scope will be discussed and addressed by the project manager and team.

13 Tools and environments

Purpose	Tool or environment	Location	Access owner	Use and backup notes
Source control	GitHub	Student Savings GitHub Repository	Joey Contreras / Whole team	Stores frontend, backend, Docker configuration, and project documents. Team members commit and push changes regularly to maintain version history.
Project board	GitHub Issues / Projects (if configured)	GitHub repository	Jonathan Ochoa	Tracks tasks, responsibilities, progress, and blockers. Update task statuses regularly and communicate changes with the team.
Development	React, Spring Boot, PostgreSQL 16, Docker Compose	Local development computers and GitHub repository	Doga Eski (frontend) / Joey Contreras (backend)	Frontend and backend are developed separately and integrated using Docker Compose. Keep configuration files and source code backed up in GitHub.

Purpose	Tool or environment	Location	Access owner	Use and backup notes
Test or demo	Local Docker Compose environment	Local environment; frontend port 5173 and backend port 8080	Whole team	Run the frontend, backend, and database together to test functionality before demonstrations. Verify that services communicate correctly and maintain test data for repeatable demonstrations.

14 Review and submission

14.1 Review decisions

Reviewer	Role	Decision	Conditions or comments	Date
[Name]	Project sponsor	[Approve / Revise]	[Comments]	[Date]
Shah, Syed Jawad Hussain	Instructor	[Approve / Revise]	[Comments]	[Date]
Lupe Campos Soto	Project documentation lead	Revised	[Comments]	11/9/2026

14.2 Final submission checklist

[ ] The plan agrees with the charter, requirements, architecture, and course calendar.

[ ] Mandatory work and the MVP appear before optional or stretch work.

[ ] Every sprint has a clear goal and demonstrable outcome.

[ ] The WBS reaches schedulable work packages with owners and completion tests.

[ ] Effort, duration, dependencies, capacity, and planned absences are realistic.

[ ] Testing, integration, documentation, reviews, and defect work appear in the plan.

[ ] Communication, risk, change, configuration, and progress practices are clear.

[ ] The plan records what changed after the latest sprint review or retrospective.

[ ] All bracketed prompts and unused example rows have been removed.

[ ] The version, date, author list, and approval decisions are current.

Recommended filename: TeamName_ProjectPlan_v1.0.docx